

CSCI 240 -- PA 2

Algorithm Analysis, Recursion, Arrays, & Linked Lists

Feel free to discuss and help each other out but does not imply that you can give away your code or your answers! Make sure to read all instructions before attempting this lab. You cannot work with a lab partner for this assignment.

You must use an appropriate provided template from Canvas and output "Author: Your Name(s)" for all your programs. If you are modifying an existing program, use "Modified by: Your Name(s)".

Review arrays and linked lists. Review recursion and how to analyze run times with big-O notation.

Exercise 1: Implement the two algorithms for prefix averages from your textbook. Run your program with the $n = 10$ values (31 -41 59 26 -53 58 97 -93 -23 84) as one test case to confirm that it is working correctly and output the prefix averages to the screen. In addition, run your program and output only the running times in ms or ns for $n = 100$, 1000, 10000, and 100000 elements (generate random values between $-n$ and n ; if you have a very fast computer, you might want to try 1000000 values). Do confirm that the collected running times match the expected running times in Big-O notation of $O(n^2)$ and $O(n)$ for the two algorithms, prefix averages 1 and prefix averages 2.

Exercise 2: Suppose you are given an array, A , containing n distinct integers that are listed in increasing order. Given a number k , provide a recursive function/method to find two integers in A that sum to k , if such a pair exists. For example, if $A = \{3, 9, 12, 15, 16, 23\}$ and $k = 25$ it outputs values 9 and 16. On the other hand, if $k = 16$ it outputs no pair exists. Output the results and the number of recursive calls for those 2 cases. *Hint: the recursive solution starts with the first and last value of the array and ignores the first or last value as applicable.*

Exercise 3 – Provide an array implementation to maintain a list of names. The following operations are available: insert rear, insert at a certain index, remove rear, remove an element at a certain index, and print the whole list. **DO NOT** implement an ADT for this assignment (i.e., do not use a class with data and operations). Just set up a fixed size array of 10 elements (will not insert more than 10 elements) and provide code in main calling functions/static methods to perform insert, remove, and print. You can set up a menu or hard code some test cases to test your implementation, but make sure to run the test cases below. Feel free to use the provided code from textbook as needed.

```
// setup global/static variables for the array and numElems below
```

```
// setup required functions/methods below
```

```
// some valid operations in main
insertRear("Jo");           // update array and numElems
insertRear("Jane");
insertAt(1, "John");
insertRear("Kim");
// display numElems           // output 4
removeAt(0);
removeRear();
printAll();                  // output John Jane
```

Exercise 4 – Provide a linked list implementation using only the head of the list and numElems to maintain a list of names (do not use a variable to refer to the tail of the list). The following operations are available: insert rear, insert front, remove a particular element, and print the whole list. Do not implement an ADT for this lab (i.e., do not use a class with data and operations). Just set up a node and use head pointer/reference to maintain a linked list of nodes and provide code in main calling functions/static methods to perform insert, remove, and print. You can set up a menu or hard code some test cases to test your implementation, but make sure to run the test cases below. Feel free to use the provided code from textbook as needed.

```
// setup Node below

// setup global/static variables for head and numElems below

// some operations in main
insertFront("Jo");           // update linked list and numElems
insertRear("Jane");
insertFront("John");
insertRear("Kim");
// display numElems           // output 4
removeElem("Jane");          // successful
removeElem("Bob");           // unsuccessful
printAll();                  // output John Jo Kim
```

Question 1: Would it be more efficient to use a tail variable for the linked list in exercise 4? Explain.

Question 2: Perform exercise 3.5.27 in C++ book or exercise 4.5.27 in Java book; set up a table showing approximate values for requested run times for the 2 columns, second and hour.

You can earn EC for only one of the options below:

Extra Credit Option 1: Add recPrint() operation to both exercises 3 and 4 as the last step to print the list using a recursive function/method. You can submit exercises 3 and 4 with the recursive function/method.

```
// call recursive version to print
recPrint(arr, numElems);          // print array in exercise 3

recPrint(head);                  // print linked list in exercise 4
```

Extra Credit Option 2: Given the integer array x with n elements, compute the maximum sum found in any contiguous sub-array **using a running time of $O(n^3)$**
solution (i.e., use a triple nested loop with outer most loop i from 0 to $n - 1$, middle loop j from i to $n - 1$, and inner most loop k from i to j ; we need to find the maximum sum from i to j). If the input array is 31 -41 59 26 -53 58 97 -93 -23 84, then the program returns the sum of $x[2]$ to $x[6]$ which is 187 (k from 2 to 6). If the input array is 31 41 59 26 53 58 97 93 23 84, then the program returns the sum of $x[0]$ to $x[9]$ which is 565 (k from 0 to 9).

Run your program with the $n = 10$ values (31 -41 59 26 -53 58 97 -93 -23 84) as one test case to confirm it is working correctly and output the sum to the screen. Run your program with $n = 10$ values (31 41 59 26 53 58 97 93 23 84) as another test case. In addition, run your program and output the sums and running times in ms for $n = 100$, 1000, and 10000 elements (generate random values between $-n$ and n). Do the collected running times match the expected running time in Big-O notation of n^3 ? Explain.

Fill out and turn in the PA submission file for this assignment (save as PDF format).